VALIDATION PLAN
INSTALLATION, OPERATIONAL AND PERFORMANCE QUALIFICATION
JR VALIDATED ENVIRONMENT — CORE

JR Validated Environment — Version 1.0

Version History

Version	Date	Author	Description
1.0	2026-03-12	Joep Rous	Initial release — v1.0 validation plan.

Approval Signatures

Role	Name	Signature	Date
Prepared by	Joep Rous	_____	2026-03-12
Reviewed by	Joep Rous	_____	2026-03-12
Approved by	Joep Rous	_____	2026-03-12

Document Information

Field	Value
Document ID	JR-VP-001
Title	Validation Plan — Installation, Operational and Performance Qualification, JR Validated Environment Core
System	JR Validated Environment
Software Version	1.0
Document Version	1.0
Status	Approved
Effective Date	2026-03-12
Organisation	Dwylup
Author	Joep Rous

Regulatory references: FDA 21 CFR Part 11 | ISO 13485:2016 | GAMP 5 | IEC 62304:2006+AMD1:2015 | EU MDR 2017/745

1. Purpose and Scope

This Validation Plan defines the approach, responsibilities, and test protocol for validating the JR Validated Environment v1.0. The JR Validated Environment provides a controlled, reproducible execution environment for R and Python scripts used in medical device development and regulatory submission workflows.

This document covers Installation Qualification (IQ), Operational Qualification (OQ), and Performance Qualification (PQ) for the core environment. Community scripts are outside the scope of this document and are covered by a separate OQ validation plan (JR-VP-002).

1.1 Regulatory Basis

This validation plan has been prepared in accordance with:

- FDA 21 CFR Part 11 — Electronic records and signatures
- ISO 13485:2016 — Quality management systems for medical devices
- GAMP 5 — A Risk-Based Approach to Compliant GxP Computerised Systems
- IEC 62304:2006+AMD1:2015 — Medical device software lifecycle processes
- EU MDR 2017/745 — EU Medical Device Regulation

1.2 Software Classification

Under GAMP 5, the JR Validated Environment is classified as Category 4 (Configured Software). Under IEC 62304, scripts executing within the environment are classified according to the safety class of the device they support. The environment itself is infrastructure software.

1.3 Scope Exclusions

- data/standard/ shared dataset library (deferred to v1.1)
- Community scripts and contrib packages (covered by JR-VP-002)
- Windows and Linux platform support (macOS only for v1.0)

2. System Description

The JR Validated Environment consists of:

- A controlled local package repository (R and Python) distributed via Dropbox
- Isolated per-project R library and Python virtual environment stored in the user home directory
- A unified wrapper script (jrrun) that enforces environment isolation, verifies project integrity, and auto-rebuilds the environment when package versions change
- Administrative scripts for installation, validation, and environment management
- A project integrity SHA-256 hash file covering all controlled scripts

2.1 Key Components

Component	Location	Purpose
jrrun	bin/	Unified wrapper — enforces isolation, integrity check, auto-rebuild
admin_install_R	admin/	Installs R packages from local repo into isolated library
admin_install_Python	admin/	Installs Python packages from local repo into virtual environment
admin_validate	admin/	Generates and runs IQ validation scripts, produces evidence file
jr_animate / jr_static	wrapper/	User entry points — thin shims calling jrrun for hello-world scripts
R_repo / Python_repo	Dropbox (external)	Controlled local package repositories — sole package source

2.2 Environment Isolation Mechanism

R packages are installed into ~/.renv/[PROJECT_ID]/library/. Python packages are installed into ~/.venvs/[PROJECT_ID]/. Both are isolated from the system R and Python installations. Direct invocation of scripts outside of jrrun is blocked by explicit environment checks at script startup.

2.3 Known Limitations and Asymmetries

Item	Description	Risk / Mitigation
R version pinning	miniCRAN --rebuild and --add fetch current CRAN versions, not pinned versions. R_requirements.txt reflects what is in the repo.	Low. The local repository is the controlled artefact. Rebuilding requires re-validation. Documented in admin manual.
Python bypass message	Python scripts fail at import time outside jrrun rather than with an explicit error referencing the wrapper.	Low. Scripts cannot run without the validated packages. Improvement deferred to v1.1.

3. Roles and Responsibilities

Role	Responsibilities	Name / Sign-off
Validation Lead	Prepares and executes validation plan, records results, authors validation report	

4. Installation Qualification (IQ)

The IQ confirms that the JR Validated Environment has been correctly installed on the target machine and that all components are present at the correct versions. The IQ is automated via the admin_validate script.

4.1 IQ Prerequisites

- macOS with Apple Silicon (arm64) processor
- R 4.5.x installed
- Python 3.11.x installed
- Dropbox synced with R_repo/ and Python_repo/ present
- admin_create_hash has been run to generate project_integrity.sha256

4.2 IQ Procedure

Step	Action	Expected Result	Actual Result
1	Run ./admin/admin_install_R from project root	All R packages installed at pinned versions in ~/.renv/[PROJECT_ID]/library/. renv.lock written. Verification passes.	
2	Run ./admin/admin_install_Python from project root	All Python packages installed at pinned versions in ~/.venvs/[PROJECT_ID]/. Verification passes.	
3	Run ./admin/admin_validate from project root	VALIDATION RESULT: PASSED. Evidence file written to ~/.jrscript/[PROJECT_ID]/validation/. R and Python versions confirmed. All packages at pinned versions.	

4.3 IQ Evidence

Attach the evidence file generated by admin_validate as Appendix A of the Validation Report. The evidence file path is:

~/.jrscript/[PROJECT_ID]/validation/validation_[DATETIME].txt

5. Operational Qualification (OQ)

The OQ confirms that the JR Validated Environment functions as designed under normal and boundary conditions. Tests include destructive actions (environment deletion, file modification) that cannot be automated and require human execution and observation.

Important: The OQ must be executed after all functional development is complete. Any functional change after OQ invalidates this plan and requires a full re-run.

5.1 OQ Test Environment

Field	Value
Hardware	MacBook Air, Apple M3, 16 GB
macOS version	macOS Sequoia 15.7.3
R version	R 4.5.0
Python version	Python 3.11.9
JR env version	v1.0 (git tag: v1.0)
Tester	[Name]
Test date	[Date]

5.2 OQ Test Protocol

ID	Description	Steps	Expected Result	Result
OQ-01	R environment rebuild from scratch	1. Delete ~/.renv/[PROJECT_ID] 2. Run admin_install_R 3. Verify output	Packages installed at pinned versions in ~/.renv/[PROJECT_ID]/library/. Verification passes.	
OQ-02	Python environment rebuild from scratch	1. Delete ~/.venvs/[PROJECT_ID] 2. Run admin_install_Python 3. Verify output	Packages installed at pinned versions. Verification passes.	
OQ-03	Run R hello-world script	1. Run: jrrun jrhello.R "Hello" 2. Verify output and plot	Script executes in validated R environment. Output and plot generated without error.	
OQ-04	Run Python hello-world script	1. Run: jrrun jrhello.py "Hello" 2. Verify output and plot	Script executes in validated Python environment. Output and plot generated without error.	
OQ-05	admin_validate full IQ validation	1. Run admin_validate 2. Verify PASSED result 3. Verify evidence file written	VALIDATION RESULT: PASSED. Evidence file generated. R and Python versions and all packages confirmed.	
OQ-06	Auto-rebuild on requirements change	1. Delete hash file in ~/.renv/[PROJECT_ID]/ 2. Run jrrun jrhello.R 3. Verify auto-rebuild triggered	jrrun detects missing hash, triggers rebuild, then executes script successfully.	
OQ-07	R --add package argument	1. Run admin_install_R --add jsonlite==2.0.0 2. Verify package added 3. Verify R_requirements.txt updated	Package added to local repo and installed. R_requirements.txt updated. Verification passes.	
OQ-08	Python --add package argument	1. Run admin_install_Python --add requests==2.31.0 2. Verify package added 3. Verify python_requirements.txt updated	Package and all transitive dependencies installed at exact pinned versions. python_requirements.txt updated.	
OQ-09	Integrity check —	1. Modify a tracked file	Step 2: PROJECT INTEGRITY	

	tampered file	(e.g. add whitespace to bin/jrrun) 2. Run jrrun jrhello.R 3. Restore file and re-run	CHECK FAILED error. Step 3: runs successfully.	
OQ-10	Bypass protection — direct script invocation	1. Run: Rscript R/jrhello.R "Hello" 2. Run: python3 Python/jrhello.py "Hello"	Both commands fail without executing in the validated environment.	

6. Performance Qualification (PQ)

The PQ confirms that the JR Validated Environment performs correctly in realistic use — running actual analysis scripts and verifying outputs against known expected results.

6.1 PQ Test Protocol

ID	Script	Procedure	Expected Result	Result
PQ-01	jrhello.R	Run: jrrun jrhello.R "Validation" and capture output	Script runs in validated R environment. Prints message and generates plot without error. Exit code 0.	
PQ-02	jrhello.py	Run: jrrun jrhello.py "Validation" and capture output	Script runs in validated Python environment. Prints message and generates plot without error. Exit code 0.	

7. Acceptance Criteria

The JR Validated Environment v1.0 is considered validated when all of the following criteria are met:

#	Criterion	Met?
1	All IQ steps pass — correct R and Python versions, all packages at pinned versions	
2	All OQ tests pass or have documented accepted findings	
3	All OQ findings are closed or formally accepted as documented limitations	
4	PQ-01 and PQ-02 pass — hello-world scripts execute correctly in validated environments	

5	IQ evidence file attached to Validation Report as Appendix A	
6	Validation Report reviewed and signed off by Validation Manager	

8. Document Control

8.1 Related Documents

Document ID	Title	Location
JR-VR-001	Validation Report — IQ/OQ/PQ, JR Validated Environment Core	docs/IQ_validation_report.docx
JR-VP-002	Validation Plan — OQ, Community Script Suite	docs/oq_validation_plan.docx
JR-OQ-001	Validation Report — OQ, Community Script Suite	docs/OQ_validation_report.docx
—	Administrator Manual	docs/admin_manual.docx
—	Software change history	CHANGELOG.md

End of Document — JR-VP-001 v1.0